

Binance Crypto Alpha Research — Project Summary

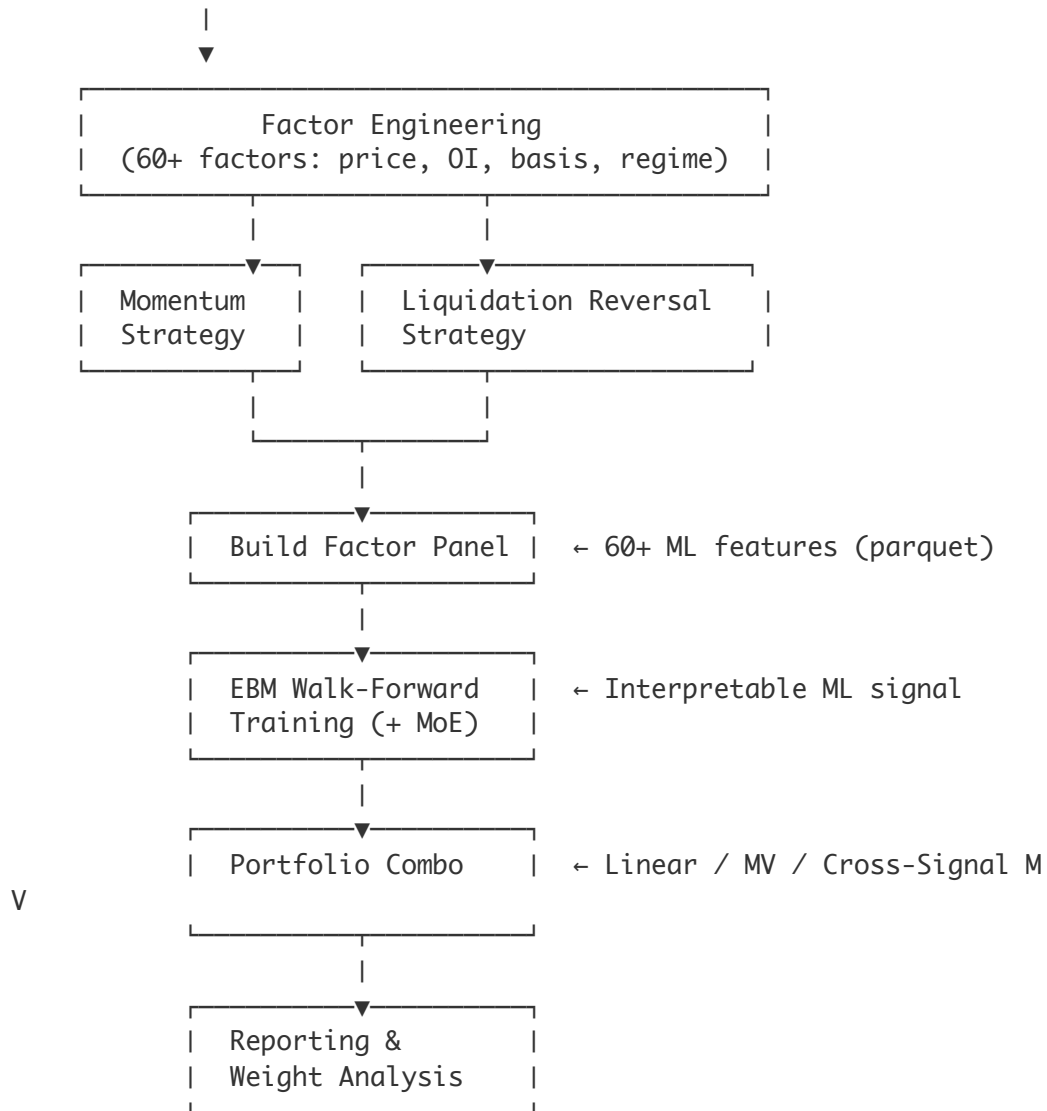
Overview

A systematic quantitative trading research pipeline built on Binance Futures data. The system spans the full quant workflow: raw data ingestion → factor engineering → strategy backtesting → ML signal training → portfolio combination → performance analysis.

Universe: ~168 perpetual futures contracts on Binance **Frequency:** Daily rebalancing **Cost model:** 10 bps commission + 5 bps slippage per trade **Position limits:** ±10% per asset, 100% gross leverage cap

Architecture

Binance Futures API + Historical Metrics (OI, LS Ratio)



Strategies

1. Momentum Strategy (`ad_mom_spot_future.py`)

Underlying Idea Exploit persistent cross-sectional momentum in crypto futures. Coins with strong price trend, growing open interest, and favorable basis positioning tend to continue outperforming. The signal is further filtered by funding rates (punish crowded longs/shorts) and volatility (shrink positions in high-vol assets).

Signal Construction

Component	Formula	Window
-----------	---------	--------

<code>price_roc</code>	30d price momentum, 10d smoothed	30d / 10d
<code>oi_roc</code>	30d OI momentum, 10d smoothed	30d / 10d
<code>trend_score</code>	$\text{price_roc} \times (1 + 2 \times \text{oi_roc})$	—
<code>basis_mom</code>	Change in (basis / price), 10d smoothed	30d / 10d
<code>vol_ratio_sig</code>	Volume ratio signal	30d
<code>sentiment_score</code>	$\text{basis_mom} \times \text{vol_ratio_sig} \times 5$, beta-neutralized	—
<code>combined_score</code>	$\text{trend_score} + \text{sentiment_score}$	—

Adjustments applied in order:

1. **Funding penalty** — boost $\times 1.5$ when funding confirms direction; kill $\times 0.5$ when funding fights position or is near-zero (no conviction)
2. **Volatility dampening** — $\text{vol_adj} = 1 - \text{vol_rank} \times 0.5$ (high-vol coins receive smaller positions)
3. **Beta neutralization** — OLS residualization removes systematic market exposure from final scores

Key parameters: `lookback=30`, `quantile=0.4`, `funding_z_threshold=1.5`, `vol_adj_factor=0.5`

2. Liquidation Reversal Strategy (`liquidation_reversal.py`)

Underlying Idea Large liquidation cascades cause temporary price dislocations. When a coin experiences an abnormally large OI drop (mass liquidation), it tends to mean-revert — particularly when the broader market regime is already stretched. The strategy captures this reversal by going against the direction of the liquidation shock.

Signal Construction

Component	Formula	Window
<code>cs_z_oi_chg</code>	Cross-sectional z-score of daily OI % change	Point-in-time
<code>ts_z_oi_chg</code>	Rolling time-series z-score of OI change	80d
<code>liquidation_shock</code>	$\max(-\text{ts_z_oi_chg} - 0.5, 0)$	—

regime_score	$(5d\ MA - 40d\ MA) / 40d\ std$, clipped ± 3	5d / 40d
interaction_alpha	liquidation_shock $\times$ regime_score	—
reversal_hawkes	EWM of interaction_alpha, half-life 12d	12d

Regime filter: Among active positions, only assets in the middle two terciles of `regime_score` are kept — avoiding both weak-signal environments and extreme trend scenarios where reversals are less reliable.

Key parameters: `half_life_decay=12`, `ts_lookback=80`,
`regime_filter_threshold=0.6`

3. EBM ML Signal (`train_ebm_signal.py`)

Underlying Idea An Explainable Boosting Machine learns non-linear relationships between 60+ cross-sectional factors and 1-day-ahead returns in a walk-forward manner. By using EBM instead of a black-box model, feature importances remain interpretable and regime-dependent behaviour can be diagnosed.

Walk-Forward Setup

Parameter	Value
Training window	252 days (rolling 1 year)
Retrain frequency	Every 21 days
Minimum training periods	126 days (6 months)
Target	<code>ret_1d</code> (raw or cross-sectional rank)
Feature normalization	Cross-sectional z-score
Embargo	1% of training window beyond target horizon

Feature groups (40+ inputs):

- **Price:** `ret_1d/5d/20d`, `volatility_30`, `beta_60`, `skewness_90`, `vol_rank_cs`
- **Momentum:** `price_roc`, `oi_roc`, `basis_norm`, `basis_mom`, `trend_score`, `sentiment_score`, `combined_score`, `funding_z`, `mom_final_score`
- **Reversal:** `ls_ratio`, `cs_z_oi_chg`, `liquidation_shock`, `regime_score`, `interaction_alpha`, `reversal_hawkes`, `rev_final_score`

- **Regime:** `volatility_regime_enc`, `trend_regime_enc`, `skew_regime_enc`, `market_adx`
- **Delta family:** 13 rate-of-change features (rolling-5 mean minus lag-10)

Post-processing:

1. Beta-neutralize raw EBM scores (OLS residualization)
2. Top/bottom quantile selection (Q=0.4)
3. Rank-proportional weight assignment

3a. Mixture of Experts Extension (`--use_moe`)

Idea: A single global model may underfit regime-specific structure. Expert EBMs are trained on the global model's residuals within each regime, capturing what the global model systematically misses.

Phase	Description
Global	EBM trained on full training window and target <code>y</code>
Residual	<code>y_residual = y_true - y_global_pred</code>
Expert	One EBM per regime, trained on <code>y_residual</code> within that regime
Inference	<code>Score = Global_Score + λ × Expert_Score</code>

Look-ahead prevention: Expert selection at time `t` uses the regime label from `t-1`, further smoothed by a hysteresis filter (default 3 days) to avoid rapid expert switching.

MoE parameters: `moe_boost_lambda=0.5`, `moe_hysteresis=3`, `regime_col=volatility_regime_enc`, `expert_interactions=5`

Factor Engineering

Factors are computed in `src/factors.py` and assembled into a flat parquet by `build_factor_panel.py`. All factors are point-in-time safe (no look-ahead).

Market Regime Signals (broadcast to all assets)

Regime	Signal	Classification
Volatility	30d rolling std, percentile	Low / Medium / High
Trend	14-period ADX on market proxy	Ranging / Weak / Strong
Skew	90d return skewness	Negative / Neutral / Positive

Regime labels are encoded numerically (`*_regime_enc`) for use as EBM features and as the MoE gating variable.

Beta Neutralization

All strategy scores and the EBM target (optionally) are residualized against a rolling 60-day market beta via cross-sectional OLS on each date. This ensures the model learns idiosyncratic alpha rather than systematic exposure.

Portfolio Combination (`backtest_combo.py`)

After individual strategies produce weight matrices, the combo optimizer blends them.

Composite Signal Construction

Before any optimization, each strategy's weight matrix is **cross-sectionally z-scored per date** before averaging, ensuring no single strategy dominates due to scale differences:

```
normed_i[t] = (w_i[t] - mean(w_i[t])) / std(w_i[t])
composite[t] = mean(normed_1[t], normed_2[t], ..., normed_n[t])
```

Combination Methods

Method	Description
<code>linear</code>	Weights $\propto$ composite alpha; normalize to max_leverage; clip per-asset
<code>equal_weight</code>	Uniform allocation; clip at max_position then scale leverage
<code>mean_variance</code>	CVXPY QP: maximize $\mu^T w - \lambda \cdot w^T \Sigma w$ subject to dollar-neutral, leverage, per-asset caps
<code>cross_signal_mv</code>	MV optimization in strategy space first (λ per strategy), then linear CS allocation

Cross-Signal Mean-Variance

Idea: Rather than blending strategies with fixed equal weights, find the time-varying allocation λ that maximizes risk-adjusted combined return:

maximize: $\mu_{\text{strat}} \cdot \lambda - \lambda_{\text{risk}} \cdot \lambda^T \Sigma_{\text{strat}} \lambda$
subject to: $\lambda \geq 0, \quad \Sigma \lambda = 1$

Where μ_{strat} = mean recent return of each strategy and Σ_{strat} = covariance of strategy returns. λ is re-estimated every day using a rolling window, so it adapts to which strategies are currently working.

Note on risk_aversion : At high values, λ converges toward equal weights (ignores noisy μ). At low values, the optimizer over-concentrates on recent winners — a manifestation of estimation error in μ over short windows.

Experiments & Analyses

Backtesting

- Individual strategy backtests with realistic costs (15 bps total)
- Walk-forward IS/OOS IC, Sharpe, total return, win rate
- IS vs OOS IC scatter per fold (overfit diagnosis)
- Quantile analysis of factor scores vs realized returns

EBM Analysis (`analyze_ebm_importance.py` , `analyze_ebm_predictions.py`)

- Feature importance ranking across walk-forward folds
- Coverage of interaction terms vs zero-boosting terms
- Raw prediction score distributions over time
- Global model vs Expert model importance comparison (MoE)

MoE Expert Usage (`analyze_moe_expert_usage.py`)

- Days active per expert, % share of total prediction window
- Number of activations, average / max run length
- Run-length distribution (violin plots per expert)
- OOS IC conditioned on which expert was active

Weight Distribution (`analyze_weight_distribution`)

- Gross leverage and net exposure time series
- Long / short asset counts over time
- Effective N (1/HHI) as diversification measure
- Daily turnover time series
- Weight magnitude histograms (long vs short side)
- Top 15 assets by average |weight|

- Portfolio breadth (21-day rolling)
- Strategy λ allocation over time (cross_signal_mv only)

Portfolio Combination

- Linear, equal-weight, mean-variance, and cross-signal MV compared
- Per-asset and per-date weight statistics saved to CSV
- Equity curves with Sharpe and total return

Output Artifacts

All outputs are saved under `./reports/strategies/{run_id}/`:

File	Content
<code>momentum.parquet</code>	Momentum strategy weight matrix (ts × symbol)
<code>reversal.parquet</code>	Reversal strategy weight matrix
<code>ebm.parquet</code>	EBM strategy weight matrix
<code>ebm_predictions.parquet</code>	Raw EBM OOS prediction scores
<code>ebm_feature_importance.csv</code>	Per-fold global EBM importances
<code>ebm_expert_importance_regime_*.csv</code>	Per-regime expert importances (MoE)
<code>ebm_report.png</code>	Feature importance, IS IC, OOS IC, cumulative PnL
<code>optimized_weights_{method}.parquet</code>	Combo weight matrix
<code>equity_{method}.csv</code>	Daily equity curve
<code>weight_stats_daily_{method}.csv</code>	Leverage, exposure, turnover per date
<code>weight_stats_assets_{method}.csv</code>	Per-asset average weights and frequencies
<code>weight_distribution_{method}.png</code>	6-panel weight analysis figure
<code>moe_expert_usage.csv</code>	Date-level expert gating log
<code>moe_expert_summary.csv</code>	Per-expert aggregate statistics
<code>moe_expert_usage.png</code>	4-panel expert usage figure

Pipeline Execution

Full pipeline (11 steps)

`./run_pipeline.sh`

Start from a specific step

`./run_pipeline.sh --start-from 8` *# Skip to factor panel building*

Run only one step

`./run_pipeline.sh --only 9` *# EBM training only*

Key individual commands

```
python -m src.scripts.train_ebm_signal \
    --run_id batch_v2 \
    --panel_path ./data/ml/factor_panel.parquet \
    --train_window 252 --retrain_freq 21 \
    --use_moe --regime_col volatility_regime_enc \
    --moe_boost_lambda 0.5 --moe_hysteresis 3
```

```
python -m src.scripts.backtest_combo \
    --run_id batch_v2 \
    --start_date 2024-01-01 --end_date 2025-12-31 \
    --strategies momentum reversal ebm \
    --method cross_signal_mv \
    --cov_lookback 60 --risk_aversion 2.0
```

Key Design Principles

1. **Zero look-ahead by construction** — walk-forward retraining, lagged regime gating, embargo gaps between train and prediction windows.
2. **Interpretability** — EBM produces per-feature importance scores rather than black-box predictions; all strategy signals are traceable to their inputs.
3. **Beta neutralization** — both training targets and prediction scores are residualized against rolling market beta, ensuring the system trades idiosyncratic alpha rather than leveraged market exposure.
4. **Modular combination** — each strategy produces a self-contained weight matrix; combination logic is fully independent and can be re-run without retraining strategies.

5. **Regime awareness** — market regimes (volatility, trend, skew) are encoded as features for the EBM and as gating variables for the MoE, allowing the system to adapt signal weights across different market environments.